



Todas as suas partes
favoritas do Medium agora
estão em uma barra lateral
para fácil acesso.

Ok, entendi

o.

n novas assinaturas por tempo limitado.



Uso de APIs OpenAI Resolvendo Problemas de Análise de Sentimento em 20 Linhas de Código

5 minutos de leitura · 9 de janeiro de 2024



Fenix Wang

Seguir



Listen



Share

... More

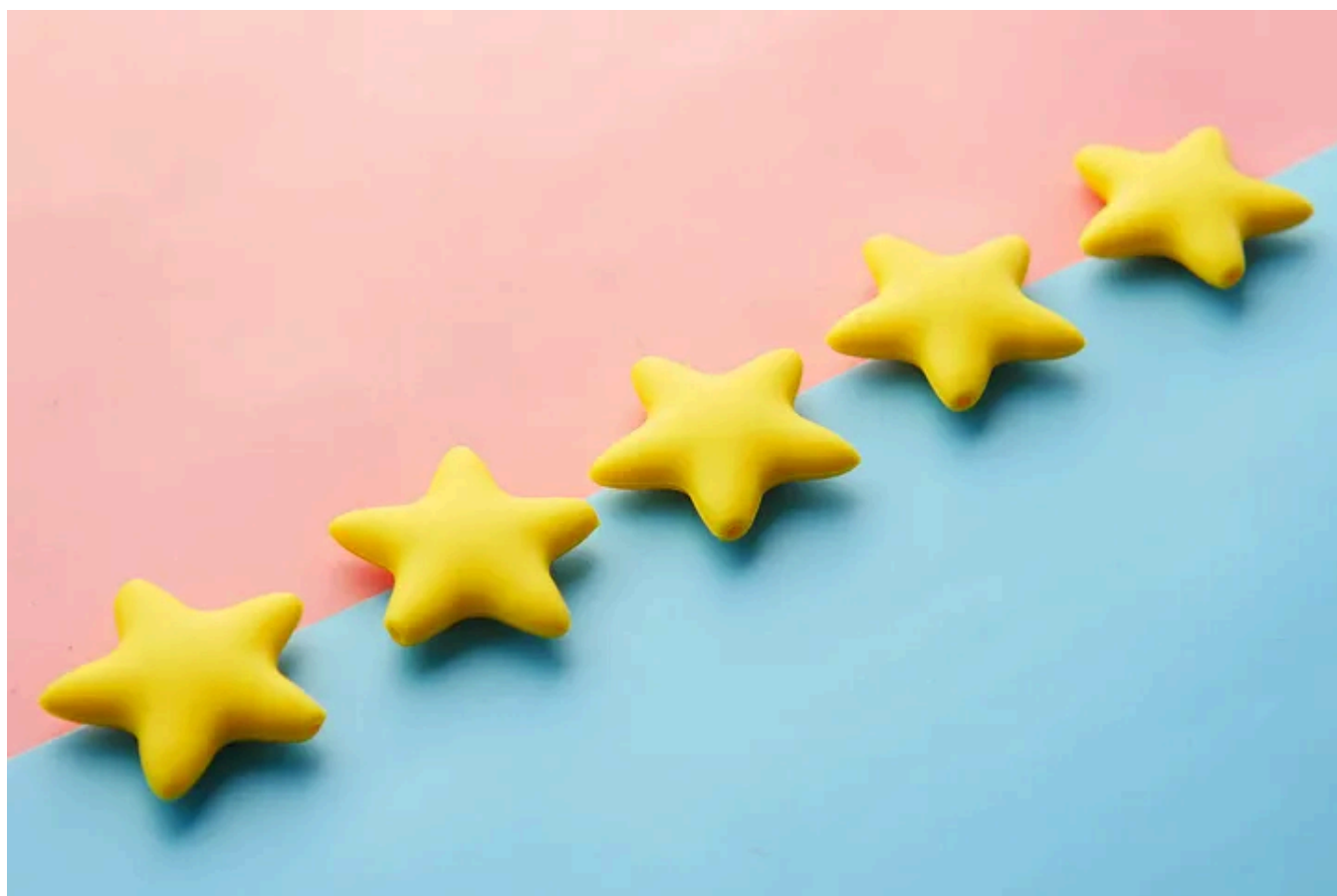


Foto por [Towfiqu barbhuiya](#) no [Unsplash](#)

A maneira mais simples de lidar com problemas de análise de sentimento usando grandes modelos de linguagem é aproveitar sua API de Embedding. Essa API pode

transformar qualquer segmento de texto que você especificar em um vetor, ou seja, um conjunto de parâmetros de comprimento fixo, para representar qualquer o grande modelo de linguagem.

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

calcular a incorporação das palavras "avaliação positiva" e API de Embedding permite obter os valores do espaço texto. Depois, a similaridade cosseno calcula a "distância" entre o texto e as palavras dadas. Ao calcular a semelhança entre o Embedding do texto e a "avaliação positiva" menos a semelhança entre esse Embedding do texto e a "avaliação negativa", obteremos uma pontuação final. Se essa nota for maior que 0, seu texto está mais próximo em "distância" de uma "avaliação positiva", então pode-se julgar que o texto provavelmente é uma "avaliação positiva"; caso contrário, pode ser uma "avaliação negativa".

Abaixo, vou usar esse método para analisar duas avaliações de brinquedos Lego da Amazon.

 mis7up

★★★★★ Great product

Reviewed in the United States on September 19, 2023

Verified Purchase

Overall, packaging was very nice. Purchased this as a gift for a kid's birthday party. Loved the detachable book on the front of the box. The kid should be extremely pleased with the item.

4 people found this helpful

Helpful

Report



Amanda

☆☆☆☆☆ Do not purchase

Reviewed in the United States on August 25, 2023

Verified Purchase

I purchased this product for a birthday gift. The Lego box was shipped in a bag and the box was all dented and damaged. No time to make a return as it was a gift.

Helpful

Report

Abaixo, vou usar esse método para analisar duas avaliações de brinquedos Lego da Amazon.

The code used for sentiment analysis is only 20 lines. Let's see if it can quickly perform sentiment analysis on these two reviews:

```
import numpy as np
from openai import OpenAI
import os
```

Todas as suas partes
favoritas do Medium agora
estão em uma barra lateral
para fácil acesso.

```
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
EMBEDDING_MODEL = "text-embedding-ada-002"
def get_embedding(text, model):
    place("\\n", " ")
    embeddings.create(input=[text], model=model).data[0].embedding
    def cosine_similarity(a, b):
        return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
    # Positive Review's Embedding
    positive_review_embedding = get_embedding("Positive Review", EMBEDDING_MODEL)
    # Negative Review's Embedding
    negative_review_embedding = get_embedding("Negative Review", EMBEDDING_MODEL)
    # Positive Example
    positive_example = get_embedding(
        "Overall, packaging was very nice. Purchased this as a gift for a kid's birthday."
    )
    # Negative Example
    negative_example = get_embedding(
        "I purchased this product for a birthday gift. The Lego box was shipped in damaged condition."
    )
    def get_score(sample_embedding):
        return cosine_similarity(sample_embedding, positive_review_embedding) - cosine_similarity(
            sample_embedding, negative_review_embedding
        )
    positive_score = get_score(positive_example)
    negative_score = get_score(negative_example)
    print("Positive rating : %f" % (positive_score))
    print("Negative rating : %f" % (negative_score))
```

Result:

```
Positive rating : 0.080185
Negative rating : -0.054586
```

As we expected, the positive review of the product obtained a score greater than 0 through the Embedding similarity calculation, while the negative review scored less than 0.

Isn't this a particularly simple way? Let me take the previous sentences evaluating coffee as an example to see if it works equally well.

```
import numpy as np
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
EMBEDDING_MODEL = "text-embedding-ada-002"
def get_embedding(text, model=EMBEDDING_MODEL):
    text = text.replace("\\n", " ")
    embeddings.create(input=[text], model=model).data[0].embedding
```

```

    return client.embeddings.create(input=[text], model=model).data[0].embedding

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

```

Todas as suas partes
favoritas do Medium agora
estão em uma barra lateral
para fácil acesso.

```

view And Negative Review's Embedding
get_embedding("Positive Review")
get_embedding("Negative Review")
get_embedding(
    this cafe is exceptional, hardly disappointing.")
get_embedding(
    "The coffee at this cafe is disappointing, hardly exceptional.")
def get_score(sample_embedding):
    return cosine_similarity(sample_embedding, positive_review) - cosine_similarity(
        sample_embedding, negative_review)
positive_score = get_score(positive_example)
negative_score = get_score(negative_example)
print("Positive rating : %f" % (positive_score))
print("Negative rating : %f" % (negative_score))

```

Result:

```

Positive rating : 0.040689
Negative rating : -0.116913

```

Similarly, we got the right results.

Real Examples on Larger Datasets

The above examples seem to work well. Could this be a coincidence? Let's take a real dataset to verify and use the OpenAI Embedding API for sentiment analysis to see if we can get the expected results.



The following code is from a sample in the OpenAI Cookbook. It uses the same method to judge user reviews of some foods Amazon provides. In this review data, tent and the star ratings users give to these foods. The star tly reflect whether our sentiment analysis method is accurate. ith 1–2 stars as negative and 4–5 stars as positive.

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

t with a CSV extension into memory using Pandas, and to avoid unnecessary API call consumption, the values that have been converted to Embedding vectors are saved in this dataset without recalculating.

```
import pandas as pd
import numpy as np

from sklearn.metrics import classification_report
datafile_path = "./data/fine_food_reviews_with_embeddings_1k.csv"
df = pd.read_csv(datafile_path)
df["embedding"] = df.embedding.apply(eval).apply(np.array)
# convert 5-star rating to binary sentiment
df = df[df.Score != 3]
df["sentiment"] = df.Score.replace({1: "negative", 2: "negative", 4: "positive", 5: "positive"})
```

For each review, I will compare it with a pre-defined “positive review” and “negative review” using the previous method and then see which review is closer in “distance” to the “positive review” or “negative review.” The “positive review” and “negative review” label texts I defined here are slightly longer, which are “An Amazon review with a negative sentiment.” and “An Amazon review with a positive sentiment.” respectively.

After calculating the results, I use the scikit-learn machine learning library to compare the predicted values with the actual user star rating data and then output the comparison results.

```
from sklearn.metrics import PrecisionRecallDisplay

def evaluate_embeddings_approach(
    labels = ['negative', 'positive'],
    model = EMBEDDING_MODEL,
):
    label_embeddings = [get_embedding(label, model=model) for label in labels]
```

```
def label_score(review_embedding, label_embeddings):
    return cosine_similarity(review_embedding, label_embeddings[1]) - cosine_similarity(review_embedding, label_embeddings[0])

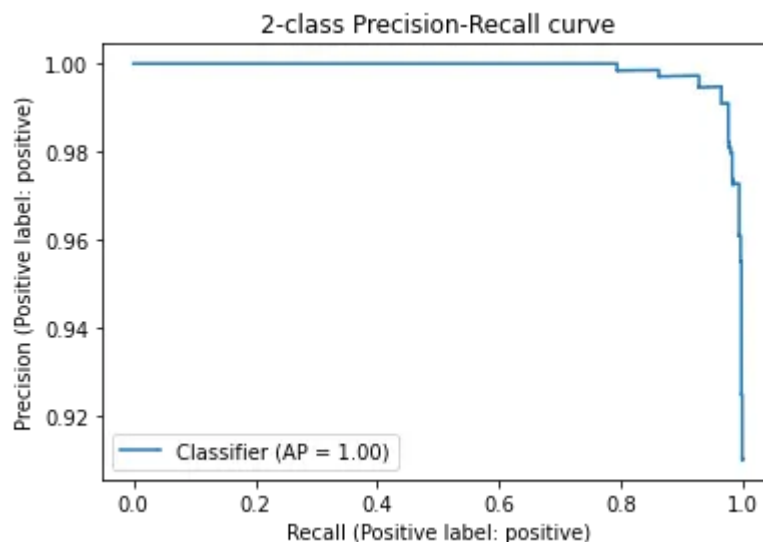
probas = df["embedding"].apply(lambda x: label_score(x, label_embeddings))
df["sentiment"] = df["embedding"].apply(lambda x: 'positive' if x>0 else 'negative')
classification_report(df.sentiment, probas)

PrecisionRecallDisplay.from_predictions(df.sentiment, probas, pos_label='positive')
_.set_title("2-class Precision-Recall curve")
_.s_approach(labels=['An Amazon review with a negative sentiment', 'An Amazon review with a positive sentiment'])
```

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

Result:

	precision	recall	f1-score	support
negative	0.98	0.73	0.84	136
positive	0.96	1.00	0.98	789
accuracy			0.96	925
macro avg	0.97	0.86	0.91	925
weighted avg	0.96	0.96	0.96	925



From the results, I can see that through this simple way to judge positive and negative reviews, the positive and negative precision reached 0.98 and 0.96, respectively, over 95%.

In terms of recall, which is recall in the chart above, the performance on negative reviews is slightly worse, only 73%, indicating that there are still quite a few negative reviews judged as positive reviews. However, the recall rate for positive reviews

reached 100%, meaning the model found all positive reviews. The overall accuracy rate is 96%, considered very high in machine learning. To achieve such accuracy, we

Todas as suas partes
favoritas do Medium agora
estão em uma barra lateral
para fácil acesso.

open API Embedding interface and add a few lines of code to
between vectors.

[https://drive.google.com/drive/1joYlOiKG3hHEVzF118i1oSvgzvgNdHHz?
usp=sharing](https://drive.google.com/drive/1joYlOiKG3hHEVzF118i1oSvgzvgNdHHz?usp=sharing)

Kaggle Example

<https://www.kaggle.com/fenixping/ai-aesthetics-chap02>

GitHub Example

[ai_aesthetics/chap02/chap02.ipynb at main · yipingw/ai_aesthetics \(github.com\)](#)

If you want to learn more, follow my book “[ChatGPT’s Guide to AI Mastery](#).”

 yipingw@outlook.com

 [@yipingw](#)

Sentiment Analysis

Artificial Intelligence

OpenAI

ChatGPT



Follow

Written by Fenix Wang

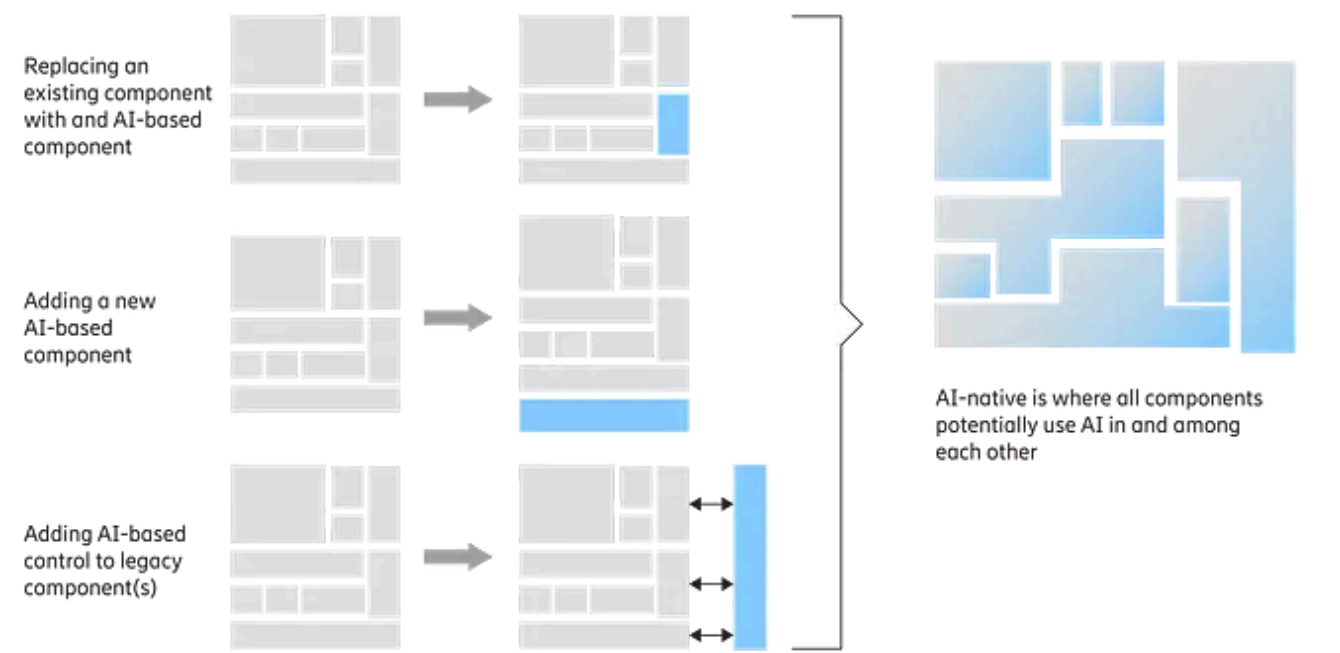
1 follower · 14 following



Todas as suas partes
favoritas do Medium agora
estão em uma barra lateral
para fácil acesso.



More from Fenix Wang



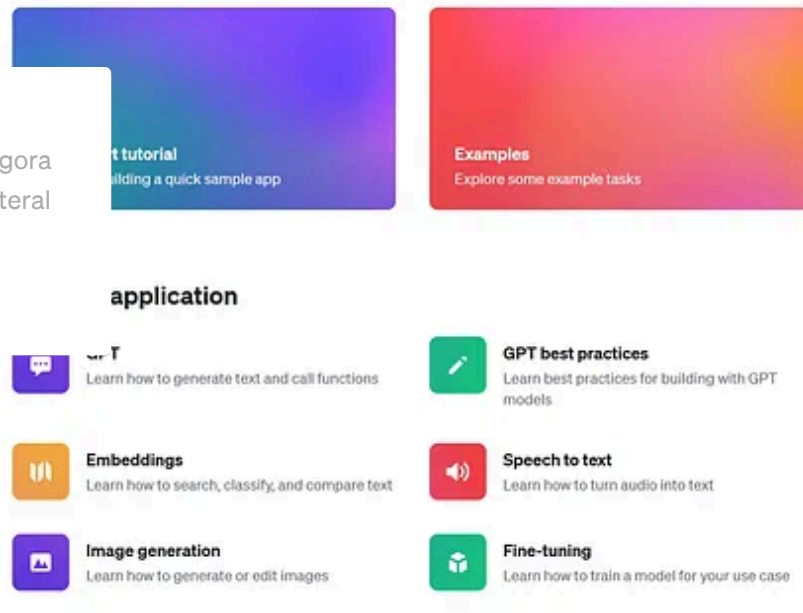
Fenix Wang

How Large Language Models to Agents are Changing the Game Rules

If large models were to be commercialized solely through providing APIs, such a model might not even be sustainable for a company like...

Start with the basics

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

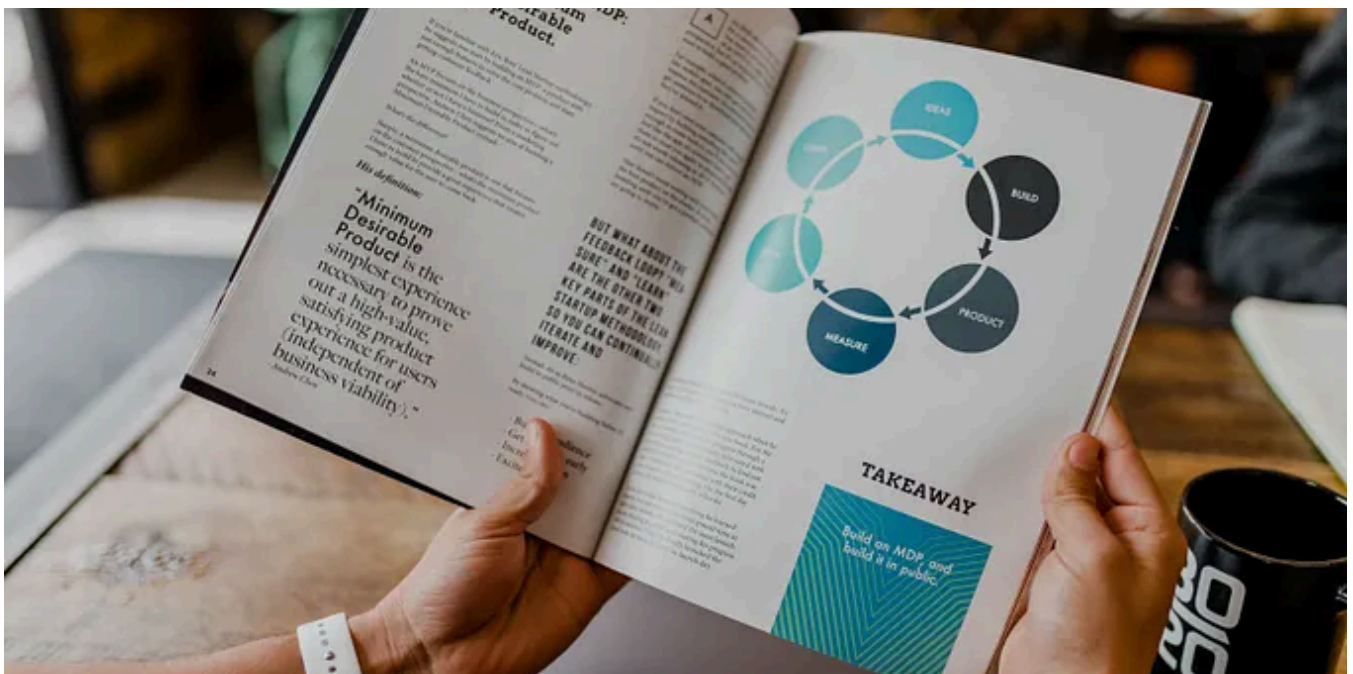


Fenix Wang

Let's Start Conversing with AI

Welcome to the world of AI! I imagine the ChatGPT craze has drawn you in—and since you're here with that curiosity, I'll deliver what...

Jan 8, 2024 6



Fenix Wang

What Exactly Are AI Agents that OpenAI's CEO is Talking About?

As one direction for AI applications to land, the concept of AI agents has received attention from many enterprises and practitioners. So...

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

See all from Fenix Wang

Recommended from Medium

:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce *Verbalized Sampling (VS)*, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., “Generate 5 jokes about coffee and their corresponding probabilities”). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: Typicality Bias Causes Mode Collapse

Tell me a joke about coffee

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

Different prompts collapse to different modes:

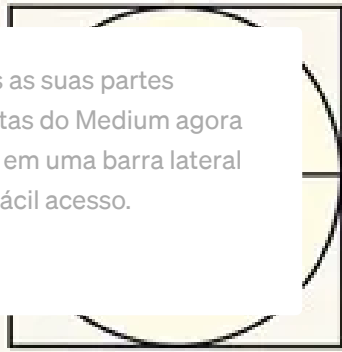
 In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can’t Believe It Worked)

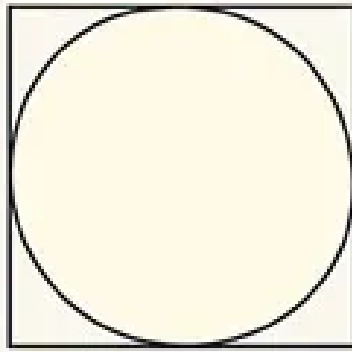
ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model — no training required...

★ Oct 19 🖱 18.6K 💬 453

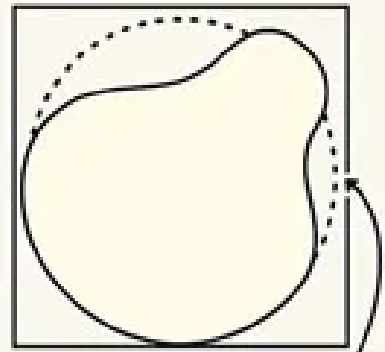
Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.



0.7854
SAU



0.74
SAU



Deviation

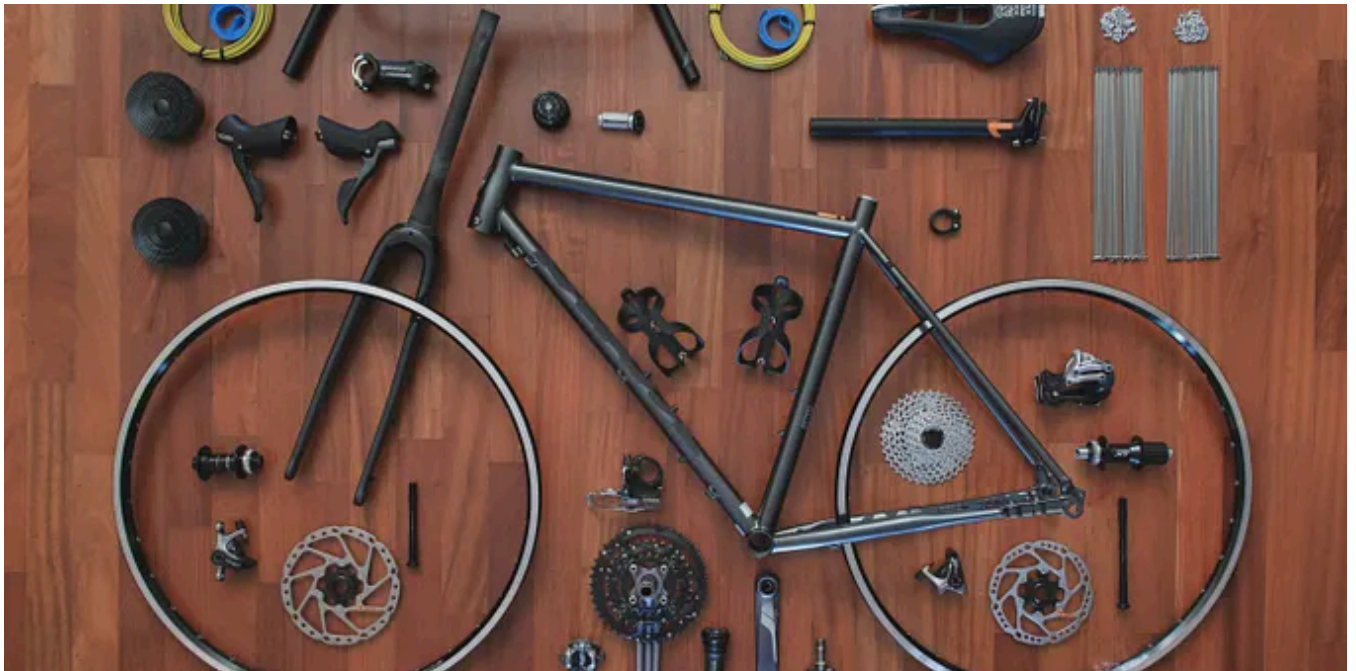


Maarten van Kroonenburgh

Reframing the Radius

Why the Geometric Ratio Model (GRM) starts with the frame, not the center

Jun 17 🖱️ 21



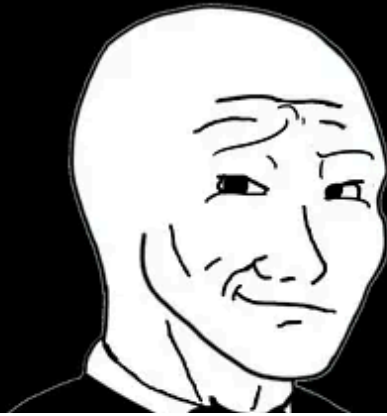
In Data Science Collective by Saurabh Sethia

Language AI : Word2Vec Deconstructed

An intuitive, step-by-step deep dive into how Word2Vec learns meaning from simple matrices, gradients, and context.

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.

Lessons Learnt building 300+ Agents



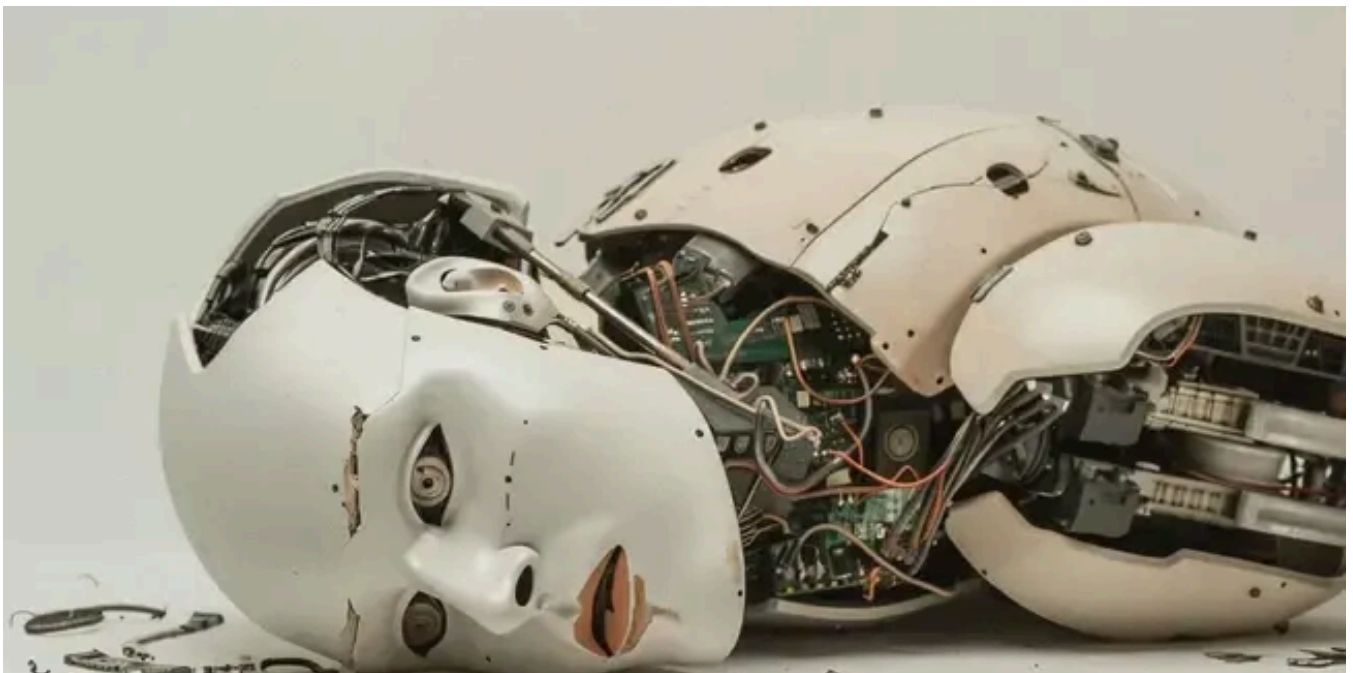
 Yashwanth Sai

I have built around 300 agents, worked at 5 startups. Here's what I learnt about AI Agent

Lessons learnt after working with agents for over an year.

★ Aug 24 5.2K 207

+

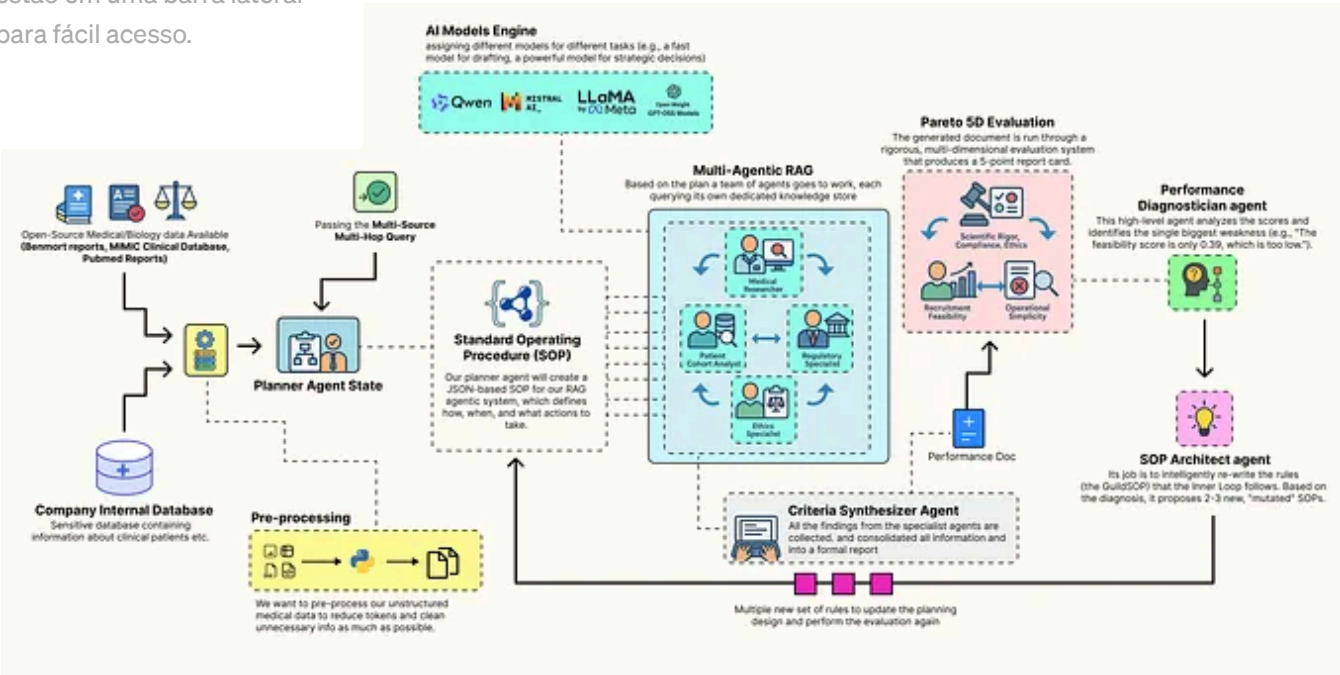


 In Predict by How to Win

MCKINSEY JUST DROPPED THEIR 2025 AI REPORT. HERE IS THE TLDR

The top 10 takeaways of the shiny new McKinsey Report on a technology promising the world, but in reality is it delivering the hype?

Todas as suas partes favoritas do Medium agora estão em uma barra lateral para fácil acesso.



In Level Up Coding by Fareed Khan

Building a Self-Improving Agentic RAG System

Specialist agents, multi-dimensional eval, Pareto front and more.

🌟 Nov 14 🖱️ 1.2K 💬 9



See more recommendations